Advanced Data Management
Quiz Refresher: Chapters 1–3, Questions

DBMS internals, storage and file organisation, indexing. Rapid recall, multiple choice with justification, and six midterm-style problems; answers in the separate answer sheet

Advanced Data Management

2026-10-27

How to use this document

Three rounds, answers in the separate answer sheet. Round A is 30 one-line recall questions: cover the answers, go fast, and mark every one you hesitate on. Round B is eight multiple-choice questions in the mock's format, where the justification is worth as much as the letter. Round C is six problems in the style of the mock's Parts 2 and 3, with numbers to compute. Budget about 25 minutes for A and B together and 60 minutes for C. Every number in the answers was computed by hand and checked against the formulas in the chapters.

Round	Content	Time
A	30 rapid-recall questions (10 per chapter)	15 min
B	8 multiple choice, letter + one-line justification	10 min
C	6 midterm-style problems	60 min
D	Answers	

Round A: rapid recall

Chapter 1: DBMS internals and the life of a query

A1. Name the five subsystems of a DBMS.
A2. Name the five stages a query passes through, in order.
A3. Which stage detects a misspelled column name, and what does it consult to do so?
A4. What does "declarative" mean for SQL, and what is the consequence for the DBMS?
A5. What three methods does every operator expose in the iterator model?
A6. What is pipelining, and which operator in a typical plan breaks it?
A7. In an `EXPLAIN` tree, which line runs first?

A8. Under a `Hash Join`, which child is the build side?
A9. What is predicate pushdown, and why does the optimiser do it?
A10. OLTP versus OLAP: one sentence each, and which storage model suits each.

Chapter 2: file organisation and storage

A11. What is the unit of cost in Chapters 2–5, and what is the page size in PostgreSQL?
A12. Why is counting pages a fair approximation of time?
A13. Name two blind spots of the page-count model.
A14. Heap file of B pages: cost of a full scan, of finding a present unique key on average, of finding an absent key.
A15. Give the formula for records per page (header 24 bytes, slot 4 bytes).
A16. Why does a slotted page store offsets in the slot directory rather than the tuples in fixed positions?
A17. Equality search cost on a heap, a sorted file, a hashed file.
A18. Why is a hashed file useless for range queries?
A19. Why do sequential reads beat random reads on a hard disk, and what does the DBMS do about it?
A20. Give two things the DBMS knows that the OS page cache does not.

Chapter 3: indexing

A21. What is an index, in one sentence?
A22. Define selectivity.
A23. Why does having an index not guarantee the optimiser uses it?
A24. Height of a B+ tree with fanout f over N entries; cost of a point lookup.
A25. Why are B+ tree leaves linked?
A26. What happens when you insert into a full leaf?
A27. Clustered versus secondary index: the difference and the consequence for range scans.
A28. State the leftmost-prefix rule.
A29. When would you use a partial index? An expression index? A covering index?
A30. Give the three costs of having an index.

Round B: multiple choice, letter plus justification

Each question is 2 points: 1 for the letter, 1 for a one-line justification (a number, a formula or a sentence).

B1. **Absent key.** A heap file has $B = 4{,}000$ pages and no index. Cost to establish that `id = 99` is *not* in the table?
(a) 1 (b) $\log_2 4000 \approx 12$ (c) 2,000 (d) 4,000

B2. **Records per page.** 8 KB pages, 24-byte page header, 150-byte fixed records, 4-byte slots. Records per page?
(a) 51 (b) 53 (c) 54 (d) 55

B3. **B+ tree lookup.** A B+ tree with fanout 200 indexes 5,000,000 rows. Page reads for one point lookup, including the heap row?
(a) 3 (b) 4 (c) 5 (d) 23

B4. **Leftmost prefix.** There is one index, on `(country, city)`. Which predicate cannot use it?
(a) `country = 'MA'` (b) `country = 'MA' AND city = 'Fes'` (c) `city = 'Fes'` (d) `country > 'M'`

B5. **Which stage.** `SELECT * FROM ordrs;` fails with "relation ordrs does not exist". Which stage reports it?
(a) parsing (b) binding (c) optimisation (d) execution

B6. **Column store.** A table has 30 columns of similar width and one million rows. `SELECT avg(price)` on a column store reads roughly what fraction of the pages a row store reads, before compression?
(a) the same (b) about 1/30 (c) about 1/2 (d) about 30 times more

B7. **Hash index.** A hash index helps which of these?
(a) `ts BETWEEN a AND b` (b) `ORDER BY email` (c) `email = 'x@y.com'` (d) `email LIKE 'a%'`

B8. **Sorted file insert.** Inserting one row into a sorted file of B pages costs, in the worst case:
(a) about 1 (b) $\log_2 B$ (c) up to B (d) 0, inserts are free

Round C: midterm-style problems

Problem: C1. Storage estimation [8 pts]

A table `tickets(id bigint, event_id int, seat varchar, price numeric, sold_at timestamp)` holds 3,000,000 rows. Column sizes: 8, 4, 12 (average, including its length byte), 8, 8 bytes. PostgreSQL adds a 24-byte tuple header and aligns each tuple to 8 bytes; each tuple costs a 4-byte slot; the page header is 24 bytes of an 8 KB page.

(a) Compute the tuple size, the records per page, the number of pages and the table size in MB.
(b) Give the cost in page reads of: a full scan; finding `id = 1234567` with no index (average); finding `id = -1` with no index.
(c) At 500 MB/s sequential throughput, how long does a cold full scan take?
(d) A colleague adds a fixed 100-byte `notes` column. Recompute the pages and the scan time, and state the factor by which the scan got slower.
(e) Which PostgreSQL calls check the page count empirically?

Problem: C2. Choosing a file organisation [6 pts]

For each workload, choose heap, sorted or hashed, and justify with the cost table (B pages).

(a) A product catalogue looked up only by exact `sku`; updated once a night.

(b) An append-only sensor log, read only by full scans for monthly reports.

(c) A ledger queried almost exclusively as "all entries between two dates".

(d) The log of (b), but now also queried by exact `sensor_id` thousands of times per hour. What does a real system do instead of changing the file organisation?

Problem: C3. B+ tree arithmetic [8 pts]

The `tickets` table of C1 (3,000,000 rows, 25,000 pages) gets a B+ tree on `id`. Fanout is 150 and a leaf also holds 150 entries.

(a) Height of the tree, with the calculation. Page reads for one lookup by `id`, including the heap row.

(b) A range on `id` matches 2,000 rows. Estimate the index-scan cost (descend, leaves, heap fetches) and compare with the sequential scan.

(c) Another range matches 300,000 rows (10% of the table). Same comparison. Which plan does the optimiser pick?

(d) Estimate the crossover: below what fraction of the table does the index scan win? Explain why the true crossover is even lower than your estimate.

(e) Toy insert: leaves hold 4 keys; a leaf holds (10, 20, 30, 40) and its parent is (10 | 50). Insert 25. Show the leaves and the parent afterwards.

Problem: C4. Index design [8 pts]

Same table, now with `status` (`'sold'` 95%, `'reserved'` 4%, `'refunded'` 1%) and `buyer_email`. For each pattern give the `CREATE INDEX`, the index type, and a one-line justification. Identify the pattern no index will help, and prove it with a selectivity number.

(a) `WHERE buyer_email = $1`

(b) `WHERE sold_at BETWEEN $1 AND $2`

(c) `WHERE status = 'refunded'`

(d) `WHERE status = 'sold'`

(e) `WHERE lower(buyer_email) = $1`

(f) `WHERE event_id = $1 AND sold_at > $2`

(g) `ORDER BY sold_at DESC LIMIT 10`

Problem: C5. Reading a plan [8 pts]

```
HashAggregate  (rows=5)
  Group Key: e.city
  ->  Hash Join  (rows=120)  (actual rows=410000)
```

```
      Hash Cond: (t.event_id = e.id)
      ->  Seq Scan on tickets t  (rows=3000000)
      ->  Hash
            ->  Seq Scan on events e  (rows=4)  (actual rows=9000)
                  Filter: (country = 'MA')
```

(a) Which node runs first, and which side of the join is built in memory?

(b) Why is the `country` filter below the join rather than above it? Name the technique and the stage that applied it.

(c) Identify the node whose estimate is most wrong, give the factor, and state the most likely cause and the command that fixes it.

(d) Before this plan existed, the query text had a typo: `countr = 'MA'`. Which stage would have rejected it, and why not the parser?

(e) Suppose a second session updates `events` while this query runs and one of the two changes is lost. Which subsystem is responsible?

Problem: C6. Discussion [7 pts]

(a) The OS already caches file pages. Give three reasons a DBMS manages its own buffer pool anyway, and for each say what breaks or slows down with the OS alone. [4 pts]

(b) "A B+ tree lookup is $O(\log_2 N)$, so for a million rows it costs about 20 page reads." Explain what is wrong with this sentence and give the right number. [3 pts]